

Week 2: Dive into CSS properties and designing

CSS Margin & Padding

1. Margin

Margin is the space outside the border of an element. It creates space between elements. A shorthand to set **all four sides** (top, right, bottom, left). Quickly set equal or custom spacing around a component.

- **Margin Top:** Sets the space top of the element. If you want to take the same space for elements from top, you can use the margin-top property.

```
Ex:- .container{  
    margin-top: 10px;  
}
```

- **Margin Bottom:** Sets space at the bottom of the element. If you want to take the same space for elements from the bottom, you can use the margin-bottom property.

```
Ex:- .container{  
    margin-bottom: 5px;  
}
```

- **Margin Left:** Sets space left of the element. If you want to take the same space for elements from the left, you can use the margin-left property.

```
Ex:- .container{  
    margin-left: 10px;  
}
```

- **Margin Right:** Sets space right of the element. If you want to take the same space for elements from right you can use margin-right property.

```
Ex:- .container{
```

```
margin-right: 10px;  
}
```

2. Padding

Padding is the space inside the element, between its content and border. It adds space within the element. A shorthand to set padding on **all four sides** adds equal or custom spacing **inside** an element for better readability or spacing.

- **Padding Top** : Sets space top the content, inside the element. If you want to create space inside the element from the top, you can use the padding-top property.

```
Ex:- .content{  
    padding-top: 10px;  
}
```

- **Padding Bottom**: Sets space bottom the content, inside the element. If you want to create space inside the element from the bottom, you can use the padding-bottom property.

```
Ex:- .content{  
    padding-bottom: 5px;  
}
```

- **Padding Left**: Sets space left of the content, inside the element. If you want to create space inside the element from the left, you can use the padding-left property.

```
Ex:- .content{  
    padding-left: 10px;  
}
```

- **Padding Right**: Sets space right the content, inside the element. If you want to create space inside the element from the right, you can use the padding-right property.

```
Ex:- .content{
```

```
padding-right: 10px;  
}
```

CSS Measurement Units

CSS units are used to define the size, space, or position of elements — such as width, height, padding, margin, font-size, etc. There are **various types of units** available in CSS, which can be used depending on whether you need a fixed or responsive layout.

1. **PX:** 1 pixel represents a single dot on the screen. It is an absolute unit, commonly used when you need precise and fixed sizing, such as for borders, icons, or small UI elements.

```
Ex:- .heading{  
    font-size: 16px;  
}
```

2. **REM:** rem (root em) is a relative unit based on the font-size of the root (<html>) element. It is more consistent than em because it doesn't inherit from parent elements, making layout and typography easier to manage 1rem = 16px.

```
Ex:- .heading{  
    font-size: 1.5rem;  
}
```

3. **EM:** em is a relative unit based on the font-size of the parent element. It is useful for scaling text and spacing in proportion to the parent, allowing for more flexible and responsive designs.

```
Ex:- .heading{  
    padding: 2em;  
}
```

CSS Box Model Browser

The **CSS Box Model** defines how browsers calculate an element's size by combining content, padding, border, and margin. By default, the **content-box** model sizes only the content, adding padding and border outside. The **border-box** model includes padding and border within the element's total width and height, making layout easier to manage.

1. **content-box (default):** Only the content is counted in the specified width and height. The padding and border are added outside of this size, increasing the element's total dimensions.

Ex:- .element {
 width: 200px;
 padding: 20px;
 border: 5px solid;
 box-sizing: content-box;
}

Total size = 200 + 202 (padding) + 52 (border) = 250px

2. **Border-box:** The content, padding, and border are all included within the specified width and height, making it easier to control element sizes and layouts precisely.

Ex:- .element {
 width: 200px;
 padding: 20px;
 border: 5px solid;
 box-sizing: border-box;
}

Total size stays 200px (content area adjusts automatically)

CSS background & effects

CSS background properties allow you to control the **visual background** of an HTML element using **colors**, **images**, **positions**, and **behaviors**. These are extremely useful in styling layouts and making designs more engaging.

- **Background color**

What it does: Sets the background color of an element.

Values: Color names (**red**), hex (**#ff0000**), RGB, RGBA, HSL, etc.

Ex.
.box {
 background-color: lightblue;
}

- **Background Image**

What it does: Sets one or more images as the background.

Values: url('image.jpg'), none

Ex.

```
.box {  
    background-image: url('pattern.jpg');  
}
```

• Background repeat

What it does: Controls how a background image repeats.

Common values:

- repeat (default) – Repeats both X and Y
- repeat-x – Repeats horizontally
- repeat-y – Repeats vertically
- no-repeat – No repetition

Ex.

```
.box {  
    background-image: url('tile.png');  
    background-repeat: no-repeat;  
}
```

• Background Size

What it does: Controls the size of the background image.

Common values:

- cover – Fill the entire area (may crop)
- contain – Fit within the area (may leave gaps)
- auto, 100px 200px, 50%

Ex. .box {

```
    background-image: url('hero.jpg');  
    background-size: cover;
```

```
}
```

- **Background Position**

What it does: Sets the starting position of the background image.

Values: left, right, top, bottom, center, 50% 50%, 10px 20px

Ex.

```
.box {  
    background-image: url('logo.png');  
    background-position: center center;  
}
```

- **Background Origin**

What it does: Specifies the positioning area for the background image.

Values:

- padding-box
- border-box
- content-box

Ex.

```
.box {  
    background-image: url('shape.png');  
    background-origin: border-box;  
}
```

- **Background attachment**

What it does: Controls scroll behavior of the background image.

Values:

- scroll: scrolls with content
- fixed: stays in place during scroll
- local: scrolls with the element

Ex.

```
.box {  
    background-image: url('bg.jpg');  
    background-attachment: fixed;  
}
```

- **Background Clip**

What it does: Defines how far the background extends.

Values:

- border-box
- padding-box

- content-box
Ex.
.box {
 background-color: lightcoral;
 background-clip: content-box;
}

Hover, Transition, Overflow, Border-radius

1. hover Pseudo-Class

- a. **What it does:** Applies styles **when the user hovers** over an element (like buttons or links).
- b. Often used with **color**, **background**, **transform**, **box-shadow**, etc.

Example:-

```
.button {  
    background-color:  
    blue;color: white;  
}  
  
.button:hover {  
    background-color: darkblue;  
}
```

2. Transition

- a. **What it does:** Adds **smooth animation** effects when properties change (e.g., on hover).
- b. Use it with properties like **background**, **color**, **transform**, **opacity**.

Example:-

```
.box {  
    background-color: lightgreen;  
    transition: background-color 0.3s ease;  
}  
  
.box:hover {  
    background-color: green;  
}
```

- c. If I want to optimize this syntax into a single line, is it possible to write it that way transition: [property] [duration] [timing-function] [delay].

Example:-

```
.box {  
    transition: background-color 0.4s ease-in-out 0.2s;  
}
```

3. Overflow

- a. **What it does:** Controls how content is handled when it exceeds the container's size.
b. Common values : visible , hidden, scroll, auto.

Example:-

```
.container {  
    width: 200px;  
    height: 100px;  
    overflow: auto;  
}
```

```
<div class="container">
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod...

```
</div>
```

4. Border Radius

- a. What it does: Rounds the corners of an element (div, button, image, etc.)

Syntax:- border-radius: [value];

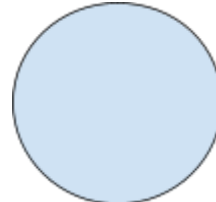
Can be in px, %, or shorthand for each corner.

Example:-

```
.box {  
    border-radius: 10px;  
}
```

```
.circle {
```

```
border-radius: 50%;  
}  
  
.custom {  
border-radius: 10px 20px 30px 40px; /* TL TR BR BL */  
}
```



External CSS

External CSS means writing your CSS code in a separate .css file instead of inside the HTML file. This file is then linked to the HTML using the <link> tag in the <head> section. Keeps styles organized and allows multiple HTML pages to share the same CSS file, making maintenance easier.

Ex:

Index.html

```
<head>
```

```
<link rel="stylesheet" href="styles.css">
```

```
</head>
```

Style.css

```
body {
```

```
background-color: lightblue;
```

```
}
```

HTML Code Comment

HTML comments are notes written in the code that do not display on the webpage. They help developers explain sections of code for better understanding and maintenance. Used to leave reminders, explanations, or temporarily disable code without deleting it.

Ex.:

```
<!-- This is a comment -->
```

```
<p>This is visible text.</p>
```

```
<!-- <p>This paragraph is commented out and won't show.</p> -->
```

CSS Code Comment

CSS comments are used to add notes or explanations within the CSS file. They are **ignored by the browser** and do not affect the layout or styles. Useful for describing sections of styles, giving reminders, or temporarily disabling code.

Ex.

```
/* This is a CSS comment */
```

```
body {
```

```
background-color: #f0f0f0; /* Light background */
```

```
}
```

```
/* h1 {
```

```
color: red;
```

```
} This is a disabled style */
```

Box-shadow & Text-shadow

box-shadow and **text-shadow** are CSS properties used to add shadow effects. **box-shadow** applies shadows to element boxes (like divs), while **text-shadow**

adds shadow to text. They enhance visual depth, highlight elements, and improve the design with simple offset, blur, and color values.

1. **Box-shadow:** box-shadow is a CSS property that adds shadow around an element's box. You can control the shadow's position, blur, and color to create depth, highlights, or floating effects in your design. It enhances the visual appearance of elements like cards, buttons, and containers.

Ex.

```
.box {  
box-shadow: 2px 4px 6px rgba(0, 0, 0, 0.3);  
}
```

box-shadow: offset-x offset-y blur-radius color;

2. **Text-shadow:** text-shadow is a CSS property used to apply shadow effects to text. It allows you to set the shadow's horizontal and vertical position, blur radius, and color. This enhances readability, adds depth, or creates glowing and stylish visual effects for headings or important text.

Ex.

```
h1 {  
text-shadow: 1px 2px 4px gray;  
}
```

text-shadow: offset-x offset-y blur-radius color;

HTML Form Elements

HTML form elements are used to collect user input. They include tags like `<form>`, `<input>`, `<textarea>`, `<select>`, and `<button>`, which enable text input, selections, and submissions. These elements allow interaction between users and websites, making them essential for tasks like login, registration, and data collection.

1. `<Form>`

The `<form>` tag defines an HTML form used to collect user input. It can contain various interactive controls like text fields, checkboxes, radio buttons, and submit buttons. Key attributes include `action` (specifies the URL where form data is sent), `method` (GET or POST), and `enctype` (defines how the form data should be encoded).

Ex. `<form action="/submit" method="POST"></form>`

2. `<input>`

The `<input>` tag is the most commonly used form element and is used to accept user input. It supports various types such as text, email, password, checkbox, radio, file, and submit. Key attributes include type, name, value, placeholder, required, readonly, and disabled, making it versatile for different input needs.

Ex. `<input type="text" name="username" placeholder="Enter your name" required>`

3. `<textarea>`

The `<textarea>` tag is used for multi-line text input, allowing users to enter longer messages or comments. Unlike the `<input>` tag, it has both opening and closing tags. Common attributes include rows, cols for size, placeholder for hint text, and maxlength to limit character input.

Ex. `<textarea name="message" rows="4" cols="50"></textarea>`

4. `<label>`

The `<label>` tag defines a text label for form elements and enhances accessibility by associating descriptive text with input fields. Using the `for` attribute, it links to the id of a corresponding input, allowing users to click the label to focus or activate the input field.

Ex. `<label for="email">Email:</label>
<input type="email" id="email" name="email">`

5. `<fieldset>`

The `<fieldset>` tag is used to group related form elements together, helping to organize forms visually and semantically. It is commonly paired with the `<legend>` tag to provide a title for the group. By default, it displays a border around the grouped fields, improving form structure and readability.

Ex. `<fieldset>
<legend>Personal Info</legend>`

```
<input type="text" name="first_name">  
</fieldset>
```

6. <legend>

The <legend> tag defines a caption or title for a <fieldset> element. It provides context for the grouped form controls, making the form more understandable and accessible. Placed at the top of the <fieldset>, it helps users quickly identify the purpose of the enclosed fields.

Ex. <legend>Contact Details</legend>

7. <select>

The <select> tag creates a dropdown menu in a form, allowing users to choose one (or multiple) options from a list. It is used in combination with <option> to define the choices and <optgroup> to group related options. It enhances user interaction by providing a compact selection interface.

Ex. <select name="country">
 <option value="india">India</option>
</select>

8. <optgroup>

The <optgroup> tag is used within a <select> element to group related <option> tags together. It adds visual and semantic structure to dropdown menus, making them easier to navigate, especially when there are many options. The label attribute is used to name each group of options.

Ex.
<select name="car">

 <optgroup label="Electric">

 <option value="tesla">Tesla</option>

 </optgroup>

</select>

9. <option>

The `<option>` tag is used within a `<select>` or `<optgroup>` element to define individual choices in a dropdown menu. Each `<option>` represents a single item that the user can select. It can include attributes like `value`, `selected`, or `disabled` to control its behavior and display.

Ex. `<option value="usa">United States</option>`

10. `<button>`

The `<button>` tag is used inside a `<select>` or `<optgroup>` element to define a single item in a dropdown list. It represents a selectable choice for the user. Attributes like `value` specify the data sent on form submission, while `selected` marks a default choice, and `disabled` prevents selection.

Ex. `<button type="submit">Send</button>`

HTML Input Types

HTML input types specify the kind of data a user can enter in a form field. Examples include text, email, password, number, checkbox, radio, and file. Each type provides a specific input method, enhancing form usability and ensuring correct data formatting.

1. Placeholder

The `placeholder` attribute is used to display hint text inside an input or textarea field. This text helps users understand what type of information is expected. It disappears when the user starts typing. It does not submit as a value unless the user enters it manually.

Ex. `<input type="text" placeholder="Enter your name">`

2. Value

The `value` attribute sets the default or pre-filled value of an input field. It is useful for showing existing data, providing default selections, or preserving user input after form submission. This value is submitted with the form unless changed by the user.

Ex. `<input type="text" value="Wscube">`

3. Readonly

The readonly attribute makes an input field non-editable while still allowing users to see and select the content. It is commonly used when displaying information that should not be changed but might need to be copied.

Ex. `<input type="text" value="Read-only info" readonly>`

4. Disabled

The disabled attribute disables an input field, making it uneditable and preventing users from interacting with it. Disabled fields are also excluded from form submission, meaning their values won't be sent to the server.

Ex. `<input type="text" value="Disabled field" disabled>`

5. Maxlength

The maxlength attribute limits the maximum number of characters a user can enter into an input or textarea field. It helps enforce input length restrictions directly in the browser.

Ex. `<input type="text" maxlength="10">`

6. Min and Max

The min and max attributes specify the minimum and maximum allowable values for input types like number, range, date, and time. They help restrict user input within a defined range.

Ex. `<input type="number" min="1" max="100">`

7. Multiple

The multiple attribute allows users to select more than one value in input fields like file uploads or email lists. For example, it lets users upload multiple files at once or enter several email addresses separated by commas.

Ex. `<input type="file" multiple>`

8. Pattern

The pattern attribute specifies a regular expression that the input's value must match for the form to be submitted. It helps validate input format, such as enforcing specific character sets or lengths.

Ex. `<input type="text" pattern="[A-Za-z]{3,}" title="Only letters, minimum 3 characters">`

9. Required

The required attribute makes an input field mandatory. Users must fill in this field before submitting the form; otherwise, the browser will show a validation error.

Ex. `<input type="text" required>`

10. Autofocus

The autofocus attribute automatically sets focus on an input field when the page loads, allowing users to start typing immediately without clicking.

Ex. `<input type="text" autofocus>`

11. List

The list attribute links an `<input>` field to a `<datalist>` element, providing predefined options as suggestions while still allowing custom input. It enhances user experience with autocomplete functionality.

Ex.

```
<input list="colors" name="color">
```

```
<datalist id="colors">
```

```
<option value="Red">
```

```
<option value="Green">
```

```
<option value="Blue">
```

```
</datalist>
```

CSS Properties in Input Box

CSS properties help style input boxes to improve their appearance, usability, and responsiveness. You can control size, color, border, padding, font, focus effects, and more to create visually appealing and user-friendly forms.

- **width / height:** Set the size of the input box.
- **padding:** Space inside the input around the text.
- **margin:** Space outside the input box.
- **border:** Defines the border style, thickness, and color.
- **border-radius:** Makes rounded corners for softer edges.
- **background-color:** Sets the input's background color.
- **color:** Changes the text color inside the input.
- **font-family / font-size:** Customize the font style and size.
- **outline:** Controls the focus outline (often removed for custom focus styles).
- **box-shadow:** Adds shadow around the input, useful for focus effects.
- **placeholder styling (::placeholder):** Customize placeholder text color and style.
- **transition:** Smooth animation effects when input state changes.
- **cursor:** Changes mouse cursor when hovering over input.

CSS - Pseudo Classes

Pseudo-classes are keywords added to selectors that specify a special state of the selected elements. They allow you to style elements based on user interaction, position, or other conditions without adding extra classes or JavaScript.

Ex.

```
button:hover {
```

```
background-color: blue;
```

```
color: white;
```

```
}
```

CSS - Pseudo Elements

Pseudo-elements let you style specific parts of an element's content or insert new content without modifying the HTML. They target sub-parts like the first letter, first line, or let you add decorative content before or after an element.

Ex.

```
p::before {
```

```
content: "Note: ";
```

```
font-weight: bold;
```

```
color: red;  
}
```

Display Properties

The display property in CSS defines how an element is displayed in the document layout. It controls the element's box type and affects its behavior in the flow of the page, such as whether it behaves like a block, inline, flex container, grid, or is hidden.

- **none:** Hides the element completely (no space taken).

Ex. `.hide { display: none; }`

- **block:** Element takes full width and starts on a new line.

Ex. `div { display: block; }`

- **inline-block:** Flows inline but allows setting width and height.

Ex. `button { display: inline-block; width: 100px; height: 40px; }`

Google Fonts & FontAwesome

Google Fonts is a free web service that provides a huge collection of open-source fonts you can easily embed into your website to enhance typography.

Ex.

```
<!-- Link Google Font in <head> -->
```

```
<link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap"  
rel="stylesheet">
```

```
<!-- Use in CSS -->
```

```
<style>  
body {  
    font-family: 'Roboto', sans-serif;  
}  
</style>
```

FontAwesome is a popular icon library offering scalable vector icons that can be customized with CSS (size, color, shadow, etc.) and easily embedded in websites.

Ex.

```
<!-- Link FontAwesome CDN in <head> →
```

```
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css"  
rel="stylesheet" >
```

```
<!-- Use icon in HTML -->
```

```
<i class="fas fa-home"></i>
```

Full vs Fixed Width Website Design

Full Width website stretches to fill the entire width of the browser window, adapting fluidly to different screen sizes.

Ex. `.container {width: 100%; max-width: 1200px; margin: 0 auto;}`

A **fixed Width** website has a set width (usually in pixels), staying the same size regardless of the browser window size.

Ex. `.container { width: 1200px; margin: 0 auto; }`

Note: Full width designs are more flexible and responsive; fixed width designs keep consistent layout but may cause horizontal scrolling on smaller screens.

Explain Width in percentage

Using **percentage (%)** for the width property makes the element's width relative to its parent container. This approach helps create fluid and responsive layouts that adjust based on screen size.

Ex.

style.css

```
.box {
```

```
width: 50%; /* Takes 50% of the parent's width */
```

```
background-color: lightblue;
```

```
}
```

```
index.html
```

```
<div style="width: 800px; border: 1px solid #000;">
```

```
<div class="box">This box is 50% wide (400px)</div>
```

```
</div>
```

Advantage: Great for building **responsive designs** that adapt to different screen sizes without fixed dimensions.

CSS Flex Container Properties

- **flex-direction:** flex-direction sets the direction of flex items in a container: row (default), row-reverse, column, or column-reverse.

Ex. `.container { display: flex; flex-direction: row; }`

- **flex-wrap:** flex-wrap controls if flex items stay on one line (nowrap) or wrap onto multiple lines (wrap, wrap-reverse) in the container.

Ex. `.container { display: flex; flex-wrap: wrap; }`

- **flex-flow:** A shorthand for flex-direction and flex-wrap.

Ex. `.container { display: flex; flex-flow: row wrap; }`

- **justify-content:** justify-content aligns flex items along the main axis (horizontally by default). It controls spacing between items using values like flex-start, center, flex-end, space-between, space-around, and space-evenly to distribute items within the flex container.

Ex. `.container { display: flex; justify-content: space-between; }`

- **align-items:** align-items aligns flex items along the cross axis (vertically by default) on a single line. It uses values like stretch (default), flex-start, center, flex-end, and baseline to control item positioning within the flex container.

Ex. `.container { display: flex; align-items: center; }`

- **align-content:** align-content aligns multiple flex lines along the cross axis (when flex-wrap is enabled). It manages spacing between lines using values like flex-start, center, flex-end, space-between, space-around, and stretch (default).

Ex. `.container { display: flex; flex-wrap: wrap; align-content: space-around; }`

CSS Flex Items Properties

- **order:** Controls the order of flex items; lower values appear first (default is 0).

Ex. `.item { order: 2; }`

- **flex-grow:** Defines how much a flex item grows relative to others when extra space is available.

Ex. `.item { flex-grow: 3; }`

- **flex-shrink:** Defines how much a flex item shrinks relative to others when space is limited.

Ex. `.item { flex-shrink: 0; }`

- **flex-basis:** Sets the initial main size of a flex item before growing or shrinking.

Ex. `.item { flex-basis: 150px; }`

- **flex:** Shorthand for flex-grow, flex-shrink, and flex-basis.

Ex. `.item { flex: 2 1 100px; }`

- **align-self:** Overrides container's align-items for individual flex item alignment on the cross axis.

Ex. `.item { align-self: center }`